CG4002 Computer Engineering Capstone Project

Design Report

Group **B03**	Name	Student #	Primary Component	Secondary Component
Member #1	Goh Sze Kang	A0271831H	Hw Sensors	Comms
Member #2	Dao Trong Khanh	A0255826W	Sw Visualizer	Sw/Hw AI
Member #3	Claribel Ho Jia Huan	A0257431H	Comms	Sw Visualizer
Member #4	Ng Chee Fong	A0273569M	Sw/Hw AI	Hw Sensors

Table of Contents

Section 0 Introduction ***4***

Section 1 System Functionalities ***5***

Section 2 Overall System Architecture ***5***

2.1 System Architecture 5

2.2 Intended Final Form 6

2.3 AR Algorithm 7

Section 3 Hardware Sensors ***8***

3.1 Introduction 8

3.2 Design choices 8

3.2.1 Sensors and algorithms 8

3.2.1 Actuators and control algorithms 10

3.3 Schematics and Pinouts 11

3.4 RTOS Mapping 11

3.5 Power and Safety Considerations 12

3.6 Assumptions, Constraints, and Risks 12

3.7 Summary of Components 13

Section 4 Communications ***14***

4.1 Overview of Communication System 14

4.2 Primary Communication Protocol: Message Queuing Telemetry Transport (MQTT) .. 14

4.2.1 MQTT Topics 15

4.2.2 MQTT Quality of Service (QoS) 15

4.2.3 MQTT Packet Format 16

4.3 Cross-Device Communication 16

4.3.1 MQTT Connection Setup (Handshaking) 16

4.3.2 FireBeetle and Laptop 17

4.3.3 Ultra96 and Laptop 17

4.3.4 Ultra96 and Visualizer 18

4.4 Secure Communication: MQTT over Transport Layer Security (TLS) 18

4.5 Handling Reliability Issues 19

4.6 Handling Concurrent Communication 19

Section 5 Software/Hardware AI **19**

5.1 Ultra96 synthesis and simulation setup 20

5.2 Neural Network Explored 20

5.3 Training and testing of model 21

5.4 Implementation of Neural Network Model on FPGA 22

5.5 Evaluation of Timing, Power and Area of Design 22

Section 6 Software Visualizer (Unity-based AR Application) **23**

6.1. Visualizer Design 23

6.2 Phone Placement, Displayed Data, and Design Constraints 24

6.3 Communication and Data Flow (Aligned with Section 4 Comms) 24

6.4 Visualizer Software Architecture (Frameworks, Modules, and Streamed Inputs) 25

6.5 Overlaying AR Information on the Camera Feed 25

6.6 Enabling AR of Gameplay Events (Serve, Hit, Return, Scoring).....25
6.7 Implemented Gameplay Rules (Scoring Scope for Automated Officiating).....26
Section 7 Project Management Plan.....27
References.....28

Section 0 Introduction

This section was a joint effort by all members.

Pickleball has been an up and coming sport since the COVID-19 pandemic, with the sport first gaining popularity in the USA to booming in popularity in Southeast Asia recently (Flamm, 2025). This growth can be seen by the rise in demand for space and facilities, with monthly pickleball court bookings at ActiveSG rising from fewer than 20 bookings per month to around 8000 bookings per month in the first half of 2025 (Liam & Mohan, 2025). This indicates a shift of the sport transiting from a niche activity to a mainstream recreational sport.

This rapid rise, however, has outpaced the development of dedicated infrastructures for pickleball courts, creating a shortage of suitable spaces to play pickleball. This shortage results in players resorting to playing at existing badminton courts or spaces in their neighbourhood. These improvisations have led to increasing tensions in the neighborhood due to the sharp sounds produced when the plastic ball comes into contact with the racket or the floor, echoing through the Housing and Development Board (HDB) flats. This can be seen in the high number of complaints on pickleball noise, with 701 complaints over a period from January 2024 to August 2025 (Sun, 2025).

This led us to the problem statement: “How can we sustain the growth and accessibility of pickleball in densely populated Singaporean neighborhoods, while providing a noise-free, space-efficient training and gameplay experience that is not limited by the availability of pickleball courts?”

If this problem is left unresolved, the noise disputes could prevent the growth of the sport and reduce community tolerances for recreational activities in shared spaces. This project is therefore significant as it seeks to accommodate the competing needs of pickleball enthusiasts and residents. By creating a solution that reduces reliance on scarce, noise-sensitive physical courts, we can promote access to the sport without compromising the livability of the urban environment.

To address this problem, the primary objective of this project is to design a comprehensive Augmented Reality (AR) pickleball training and gameplay experience that works reliably in limited spaces and reduces reliance on physical courts. We will implement a wearable and a racket integrated with Inertial Measurement Unit (IMU) sensors to capture swing dynamics in real time. We will also develop a multi task neural network that outputs the opponent’s racket state and shot type, and deploy the forward pass on the Ultra96 using Vitis HLS fixed point inference with scaled in scaled out handling at the ARM side. Finally, we will integrate the sensing, communications, AI inference, and Unity visualiser into one end to end system, and evaluate its accuracy, end to end latency, and FPGA resource and timing feasibility under realistic gameplay conditions.

Section 1 System Functionalities

This section was a joint effort by all members.

We are proposing a single-player AR pickleball system where a player uses a smart racket and a phone visualiser to rally against an AI opponent. The system aims to provide a realistic training experience comparable to an actual pickleball game. The racket will aim to accurately track swing motion and provide tactile feedback. The phone visualiser will aim to render the AR court, ball physics and show the opponent returns to the player dynamically.

Feature	User Story
Seamless Session Initialisation	As a Player, I want the system components to pair automatically upon startup, so that I can begin training immediately.
Location Independent Gameplay	As a Player, I want to be able to play anytime, anywhere, so that I am not restricted by facility availability and timing.
Intentional Serve Trigger	As a Player, I want to initiate rallies only through a specific, intentional serve gesture, so that accidental movements or repositioning do not prematurely trigger gameplay events.
Low-Latency Motion Tracking	As a Player, I want the virtual racket to mirror my physical swing mechanics with negligible latency, so that I can practice my timing and aim with realistic visual responsiveness.
Adaptive AI Response	As a Player, I want the AI opponent to generate return shots dynamically based on my trajectory, so that I face varied and unpredictable gameplay scenarios rather than a static, scripted loop.
Haptic Hit Confirmation	As a Player, I want to receive distinct vibration feedback immediately upon contacting the virtual ball, so that I have sensory confirmation of a successful hit.
Automated Officiating	As a Player, I want the system to automatically validate shots and determine rally outcomes, so that I can continue playing fluidly without stopping to arbitrate rules manually.
Real-Time Telemetry	As a Player, I want essential game telemetry to remain visible within the AR view, so that I can track my performance contextually without breaking immersion or checking a secondary screen.

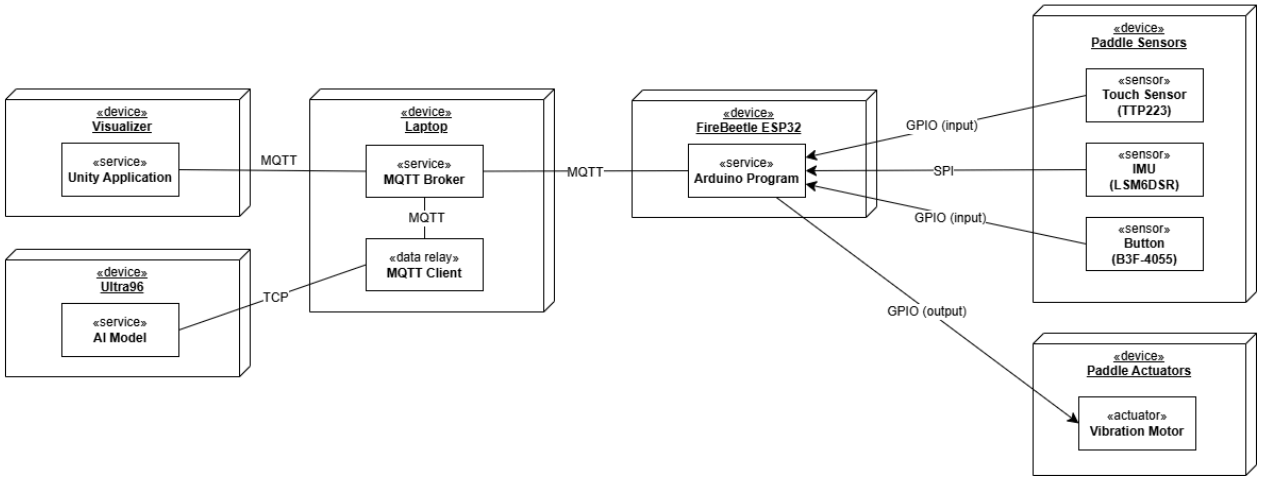
Section 2 Overall System Architecture

This section was a joint effort by all members.

2.1 System Architecture

The system architecture follows a distributed modular design connected via a central MQTT messaging backbone as shown in the diagram on the next page. The system consists of four logical nodes:

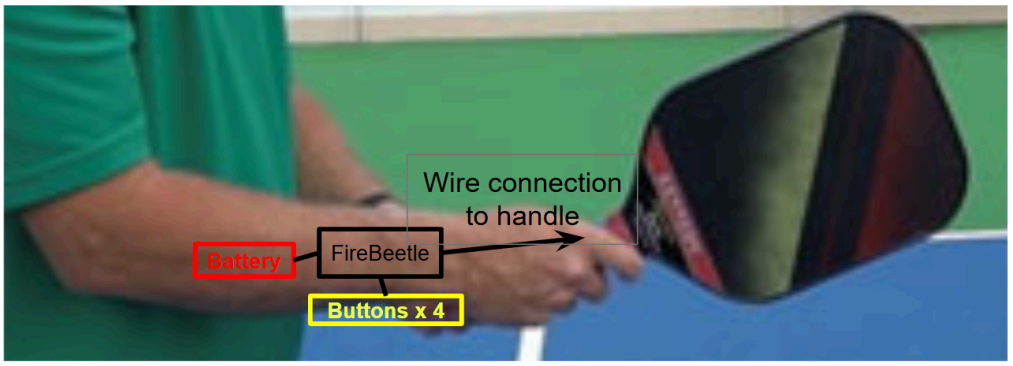
- Ultra96 - AI inference
- Laptop - Central Relay and MQTT broker
- Visualiser - AR Display
- FireBeetle ESP32 - Hardware Controller



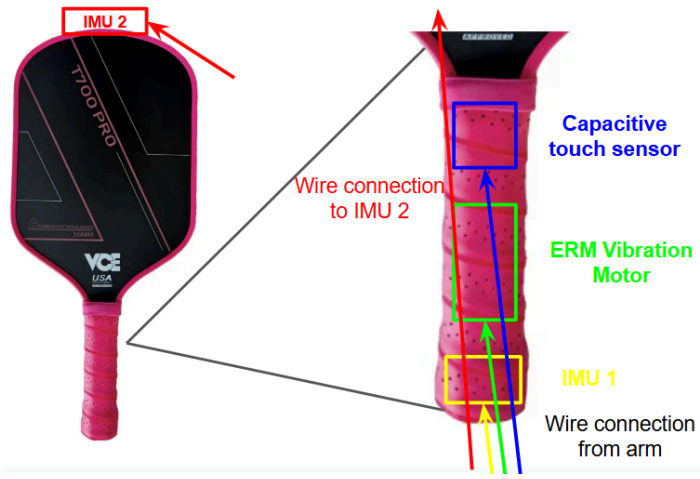
2.2 Intended Final Form

The intended final design aims to adopt a tethered wearable configuration that splits the components between a forearm-mounted control unit and the instrumented pickleball paddle.

The primary processing unit, consisting of the Firebeetle ESP32, the Li-Po battery, and the four tactile user interface buttons. These will all be housed in a wearable enclosure secured to the player's forearm in the configuration shown below. This unit will act as the central hub for power and data processing and is connected via a flexible wire harness to the racket handle. This design choice shifts the bulk of the weight to the arm, thereby reducing the weight on the racket to preserve natural swing mechanics.



The racket itself is planned to be embedded with distributed sensors to capture high-fidelity motion data as shown in the diagram below. The main bulk of the components (IMU 1, vibration motor, touch sensor) is embedded in the handle for easy feedback and interaction for the user. To capture more accurate rotational dynamics and tip velocities, a second IMU sensor is mounted at the top edge of the paddle face.

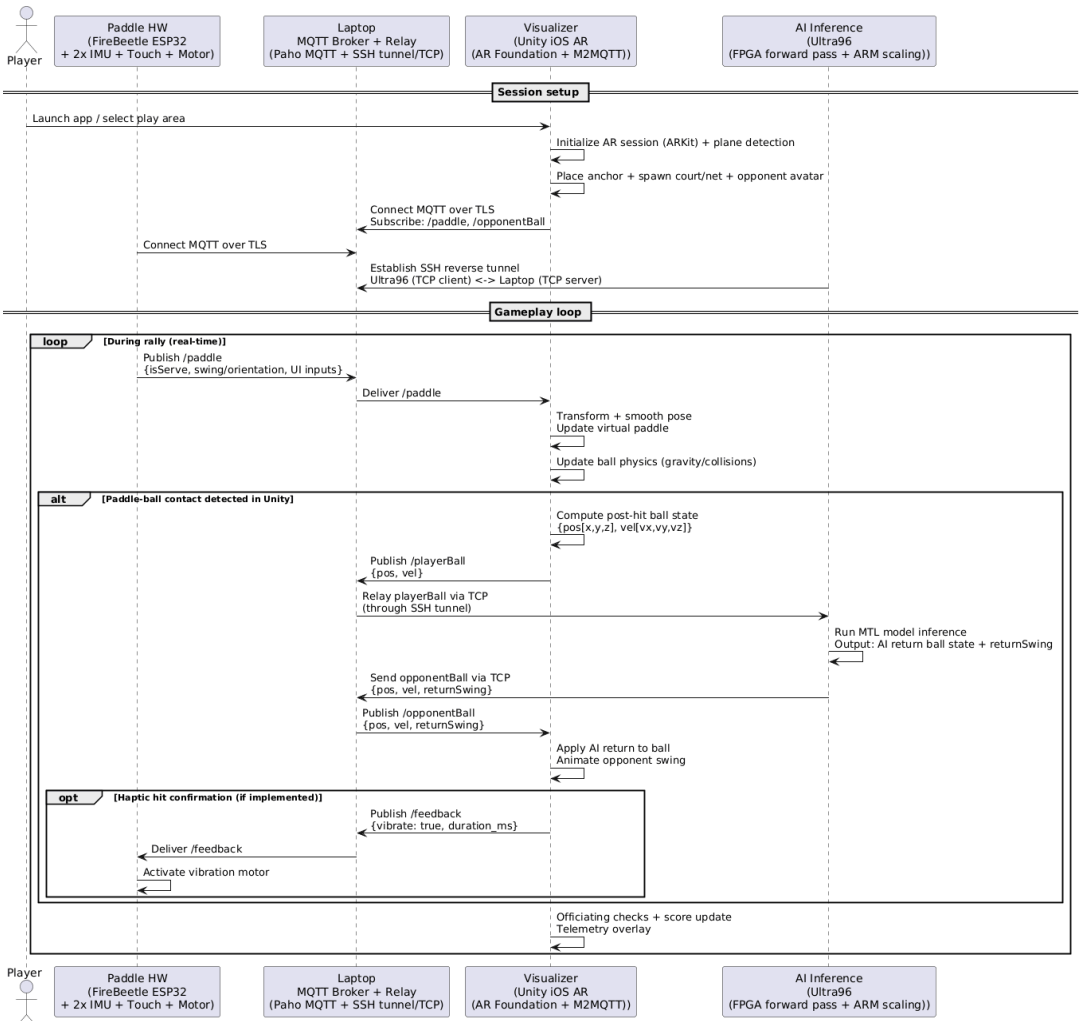


2.3 AR Algorithm

The AR algorithm supports a low-latency, first-person pickleball rally by anchoring a real-scale court in the user's environment and synchronizing paddle motion with an AI-generated opponent return. It consists of six main steps spanning the Visualizer (Unity iOS AR), FireBeetle paddle hardware, the laptop MQTT broker/relay, and Ultra96 AI inference. The execution flow is defined by the following sequence:

1. **Calibration:** The user places the racket flat on the floor at the center of the area he wants to play in to establish a zero-offset bias, aligning the hardware IMU orientation with the AR world frame before gameplay begins.
2. **Input Streaming:** The FireBeetle detects serve intent via the capacitive sensor and swing motion, publishing the filtered paddle state to the Visualizer via the `/paddle` MQTT topic.
3. **Virtual Mapping:** The Visualizer transforms the raw sensor data into the AR coordinate system and applies smoothing to update the virtual paddle's pose in real-time.
4. **Physics & Hit Export:** Unity simulates the ball trajectory and, upon paddle contact, calculates the post-hit velocity to publish on the `/playerBall` topic.
5. **AI Inference:** The Ultra96 executes the neural network forward pass on the received ball data to predict the opponent's return shot, publishing the result to `/opponentBall`.
6. **Rendering & Feedback:** The Visualizer animates the opponent's return, updates game telemetry, and triggers the racket's vibration motor to confirm the successful hit.

A UML sequence diagram summarizing the cross-device workflow (Player, Paddle HW, Laptop broker/relay, Visualizer, Ultra96) is provided as follows:



Section 3 Hardware Sensors

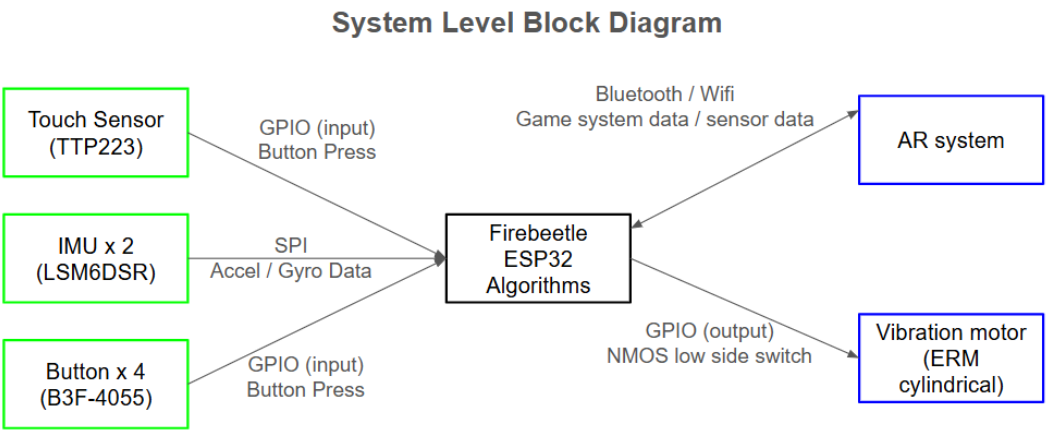
This section is done by Sze Kang.

3.1 Introduction

The system, built on a FireBeetle ESP32 running an RTOS, is a handheld, racket-mounted input and feedback device for an AR pickleball game. It uses a capacitive touch sensor and two IMUs to detect player intent and racket motion, four mechanical buttons for in-game interactions, and a vibration motor to provide haptic feedback for confirmed actions or events.

Sensor data is acquired event-drivenly by the RTOS, processed for motion detection, input validation, UI interaction, and used to control the actuator. This forms a sensor–algorithm–actuator loop: sensors capture intent and motion, algorithms validate and interpret inputs, and actuators provide feedback. The RTOS ensures each stage runs independently, maintaining smooth operation during rapid gameplay.

3.2 Design choices



3.2.1 Sensors and algorithms

Capacitive Touch Sensor (TTP223)

The TTP223 capacitive touch sensor serves as

the intentional action trigger, communicating with the ESP32 via a GPIO digital output (logic HIGH = touch event).

<u>Purpose and Selection Justification:</u>

It detects when the player intends to serve. A serve is registered only when a valid touch coincides with a forward swing detected by the IMU, reducing false triggers from normal racket movement, accidental contact, or environmental vibrations. The sensor can also be repurposed for in-game power-ups or special actions.

Sensor Type	Pros	Cons
Digital Capacitive Touch Sensor	• Immune to swing vibration • Clean digital signal (no debouncing needed → able to fit in racket handle) • Light touch → good UX when serving	• Sensitivity affected by gloves or very thick grip material • Requires calibration/tuning for moisture and environment

Piezo Touch Sensor	• Extremely thin and lightweight • No moving parts • Works through most materials	• Highly sensitive to vibration and shock (false triggers during swings) • Requires analog filtering and threshold tuning
Mechanical Button	• Simple and familiar; easy to interface • Clear tactile click	• Ergonomic conflict: Users don't want to press while swinging

Sensor Processing:
Touch events are handled via GPIO interrupts for low-latency, event-driven detection without the need for continuous polling. Touch events are logically combined with motion data to confirm valid serves.

Mechanical Buttons (4×B3F-4055)

Four tactile push buttons connect to the ESP32 via GPIO inputs with pull-up resistors, providing a reliable, low-latency interface for AR menu navigation.

Purpose and Selection Justification:

The buttons are used for user interactions such as menu navigation and option selection. Each button is mapped to a specific function:

- Up, Down, Left (Return), Right (Select)

Sensor Processing and Timing Considerations:

Button inputs are processed using GPIO interrupts for immediate response, while software debouncing is applied to eliminate false triggers caused by mechanical contact bounce.

Inertial Measurement Units (2×LSM6DSR)

Two LSM6DSR IMUs are used to capture the racket's motion and orientation. Each IMU communicates with the ESP32 via the SPI interface and provides raw 16-bit signed data from both the accelerometer and gyroscope.

- Accelerometer: measures linear acceleration along the X, Y, and Z axes
- Gyroscope: measures angular velocity about the X, Y, and Z axes

Purpose and Selection Justification:
The IMUs enable detection of swing direction, swing intensity, and racket orientation, which are essential for interpreting gameplay actions such as serving and hitting the ball. Using two IMUs improves robustness and accuracy of motion tracking.

The LSM6DSR IMU, featuring a closed-loop gyroscope and SPI interface, was selected to achieve low drift, high accuracy, and fast data acquisition. The increased wiring and implementation complexity are acceptable trade-offs given the performance requirements of the system.

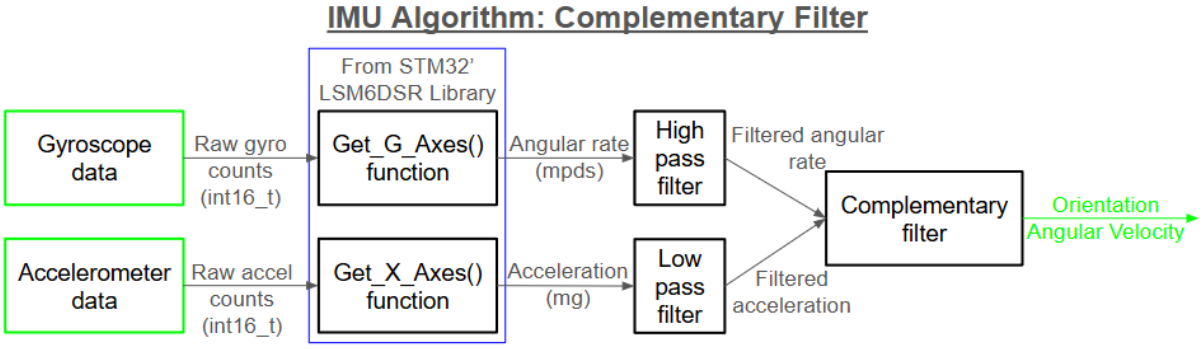
According to its datasheet, the LSM6DSR operates in the sub-milliampere range under normal sampling conditions (≈ 0.7 mA at 208 Hz), which is negligible compared to the significantly higher current consumption of the FireBeetle ESP32 (≈ 80mA). Consequently, the slightly higher power consumption of a closed-loop IMU is a worthwhile trade-off.

The IMU is designed to operate efficiently using INT1 interrupts, which signal the ESP32 when new data is available, allowing the IMU task to be event-driven. Additionally, the internal FIFO buffer stores multiple sensor samples, enabling burst reading to reduce SPI transactions and maintain accurate motion tracking even if the RTOS task experiences slight delays.

Before gameplay, a simple calibration step is performed by placing the racket flat on the floor with the paddle facing the front, establishing a reference orientation for court alignment.

Category	Options	Pros	Cons
Control Architecture	Open-loop IMU	- Simpler, cheaper and faster	- Higher drift over time - Lower long-term accuracy
	Closed-loop IMU (selected)	- Lower drift - Higher accuracy and stability	- Higher complexity - Slightly higher power consumption
Communication Interface	I2C	- Uses fewer wires	- Slower data rate - Half-duplex communication
	SPI (selected)	- Faster data transfer; lower latency - Full-duplex communication	- Requires more wires - Chip-select management needed

Sensor Processing Algorithms:



Raw accelerometer and gyroscope data are processed in real-time by RTOS tasks to estimate racket orientation and motion. A complementary filter fuses the sensors:

- Accelerometer: low-pass filtered for long-term stability
- Gyroscope: high-pass filtered for fast motion responsiveness

The fused angle is calculated as:

$$\theta_{\text{est}} = \alpha(\theta_{\text{prev}} + \omega_{\text{gyro}} \cdot \Delta t) + (1 - \alpha)\theta_{\text{accel}}$$

where $\alpha \in [0,\ 1]$ is the weighting factor that balances the short-term gyro responsiveness with long-term accelerometer stability. Combined with the capacitive touch sensor, this approach provides smooth and responsive estimates suitable for detecting swings and serves.

In addition, the linear velocity of the racket at ball contact is estimated using the angular velocity from the handle-mounted IMU combined with the known rigid-body geometry of the racket. The tip velocity is calculated as the cross product of the handle angular velocity and the vector from the handle to the contact point (center of the paddle):

$$\vec{v}_{\text{tip}} = \vec{\omega}_{\text{handle}} \times \vec{r}_{\text{handle}\rightarrow\text{tip}}$$

Filtering applied to IMU signals:

- Angular velocity: HPF removes slow wrist drift; LPF removes micro-vibrations.
- Linear acceleration: HPF removes long-term bias; LPF reduces noise before short-window integration.
- Effect: Band-pass isolates true swing motion.

This velocity is used by the AR system to determine the speed of the return ball. By using a rigid-body assumption and focusing on the brief impact interval, drift is negligible, and swing dynamics can be accurately captured.

3.2.1 Actuators and control algorithms

Vibration Motor (ERM Cylindrical Motor)

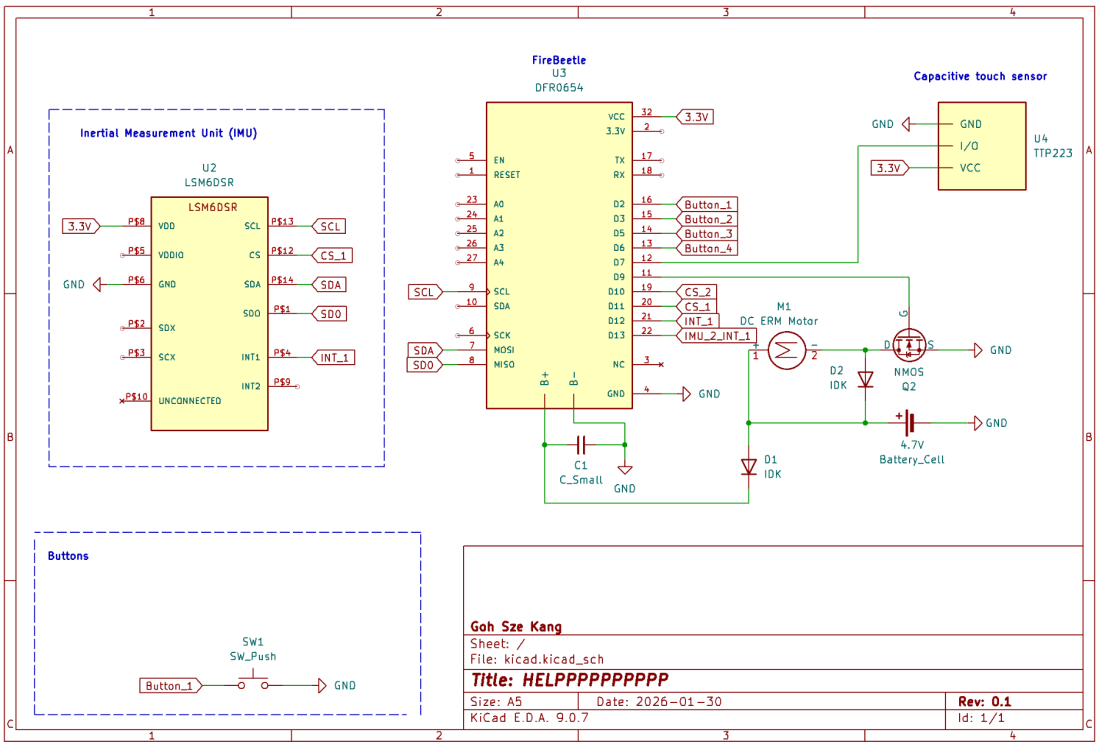
Purpose and Selection Justification:

An ERM cylindrical vibration motor is used to provide haptic feedback to the player when specific in-game events occur, such as successfully hitting the pickleball. This is done via a GPIO-controlled low-side NMOS switch to safely handle the motor's current requirements.

An ERM cylindrical motor was selected due to its strong, omnidirectional vibration, which remains perceptible through the racket handle and grip. Fragile alternatives like piezo actuators were deemed unsuitable for handheld, frequently swung use. In addition, the motor is activated only briefly during in-game events (e.g. successful hits), making its lower efficiency and slower response time acceptable trade-offs.

3.3 Schematics and Pinouts

This section presents the electrical schematics and ESP32 pin assignments for all sensors and actuators. It includes SPI connections for the IMUs (with dedicated chip-select pins and INT1 lines for event-driven data acquisition), GPIO inputs for the capacitive touch sensor and mechanical buttons (with pull-up resistors), and GPIO output control for the vibration motor via a low-side NMOS switch (including flyback diode protection). Power supply lines and decoupling capacitors are also shown.



3.4 RTOS Mapping

Task	Contents / Responsibilities	Priority / Notes
Input & Control Task	Handles GPIO inputs (touch sensor, buttons). Processes user input, debounces buttons, applies thresholding.	High - preemptive. Wakes up via GPIO interrupts
IMU Task	Reads accelerometer and gyroscope data when INT1 interrupt fires. Applies filtering and calculates swing direction/intensity.	High - event-driven, deterministic timing, avoids wasted CPU cycles.
Wireless Send Task	Sends AR system data asynchronously when new events arrive (from Input/IMU tasks).	Medium - should not block sensor tasks; event-driven.

Wireless Receive + Motor control	Polls for AR system messages. Activates ERM motor when AR system confirms events (e.g. successful ball hit).	Medium - polling rate must be fast enough for realistic haptic feedback timing.

3.5 Power and Safety Considerations

Component	Voltage	Current	Notes
FireBeetle ESP32	3.3V	≈ 80mA (peaks up to 240mA with Wi-Fi)	Lower peaks when using BT/BLE (≈ 130mA)
LSM6DSR IMUs (x2)	3.3V	≈ 0.7mA per IMU at 208Hz (1.2mA at high perf.)	Current depends on ODR and performance mode
ERM motor	3.0V	≈ 60mA (steady-state)	Intermittent operation; startup inrush current handled by NMOS and decoupling capacitors
Buttons (x4) + Touch sensor	3.3V	Negligible (≈ <1mA combined)	For each button, I ≈ (3.3V / 45kΩ) ≈ 73µA

$P_{total} = 3.3 * (0.08 + 0.0007 * 2 + 0.001) + 3.0 * (0.06) \approx 0.452$ W

$I_{total} = 80 + 0.7 * 2 + 60 + 1 \approx 142.4$ mA

For a 450mAh 3.7V 602035 LiPo battery:

$$\text{Battery life} = \frac{450mAh}{142.4mA} \approx 3.16 \text{ hrs}$$

Adjusted for Wi-Fi peaks and safety margin:

$t_{usable} \approx 2.5$hrs

With recreational pickleball games typically lasting 15-25mins, a single charge is more than sufficient for multiple sessions. The 602035 LiPo is also compact enough to fit below the FireBeetle board, optimizing space usage.

<u>Power supply considerations:</u>

The FireBeetle and the ERM motor will be powered by the 3.7 V LiPo battery, while the remaining components (IMUs, buttons, and touch sensor) will be powered by the 3V3 port on the FireBeetle. The 3V3 output is capable of providing up to 500 mA peak current, making it safe for powering these low-current components.

To manage the inrush current and electrical transients generated by the vibration motor, protective circuitry is incorporated into the motor drive design. The motor is driven using a low-side NMOS switch, allowing the ESP32 to control the motor without directly sourcing the startup current.

A flyback diode is placed across the motor to safely clamp voltage spikes caused by back electromotive force (back-EMF) when the motor is switched off. This protects the NMOS and surrounding circuitry from overvoltage stress and reduces electromagnetic interference.

Additionally, a decoupling capacitor is placed close to the ESP32 power input to buffer transient current demands and minimize voltage dips during motor activation or wireless communication bursts, reducing the risk of brown-out resets.

3.6 Assumptions, Constraints, and Risks

Category	Description	Mitigation
Assumptions	Player swings and holds the racket	Increase IMU sampling frequency and

	normally; extreme motions are rare	data transmission rate to AR system
	RTOS can handle concurrent tasks with minimal latency	Prioritize real-time tasks; perform timing tests and calibration
Constraints	Power supply is limited by battery capacity; motor and wireless current spikes must be handled	Include smoothing capacitors; design motor drive to handle inrush current
	Electrical components must fit within the racket handle	Careful mechanical design; compact components
Risks	Capacitive touch or buttons may misfire due to environment or motion	Apply debouncing, filtering, and threshold tuning
	Sensor errors such as IMU drift	Implement calibration, filtering algorithms
	Low battery capacity or sudden drops causing brown-outs	Include smoothing capacitors; design motor drive to handle inrush current

3.7 Summary of Components

Component	Type	Purpose	Interface	Key Considerations/ Limitations
Firebeetle ESP32	DFR0478	Main controller for system logic, sensor integration, and motor control	GPIO, I2C, SPI, UART	Wi-Fi/Bluetooth significantly increases power draw (peaks at 240 mA during Wi-Fi TX)
Capacitive touch sensor	TTP223	Acts as a button for serving the ball	GPIO	Susceptible to false triggers; size constraints within the racket handle
2x IMUs	LSM6DSR	Motion tracking (acceleration and angular velocity)	SPI	IMU drift over time; lack of accuracy
Vibration motor	ERM cylindrical	Provides haptic feedback to user	GPIO	Startup current spikes and back-EMF cause voltage dips and EMI; vibration strength vs power draw trade-off
Tactile switches	B3F-4055	User interaction with game interface	GPIO	Requires debouncing (hardware or software)
Tactile Switch caps	B32-1330	Improves ergonomics of switches	N/A	Loose fit possible; may detach under vibration
18650 lithium ion batteries	Panasonic NCR18650 B	Used for daily testing of components	N/A	Capacity and max discharge current
18650 battery case	NIL	Holds and supplies power from 18650 batteries	N/A	In-built switch for more control
3.7V Li-Po batteries	602035 (450mAh)	Power source for actual run	N/A	Size affects comfort; capacity must reliably power all components
Digital power monitor	INA219	Measures system voltage and current consumption	I2C	Limited current range; current spikes may exceed measurement limits

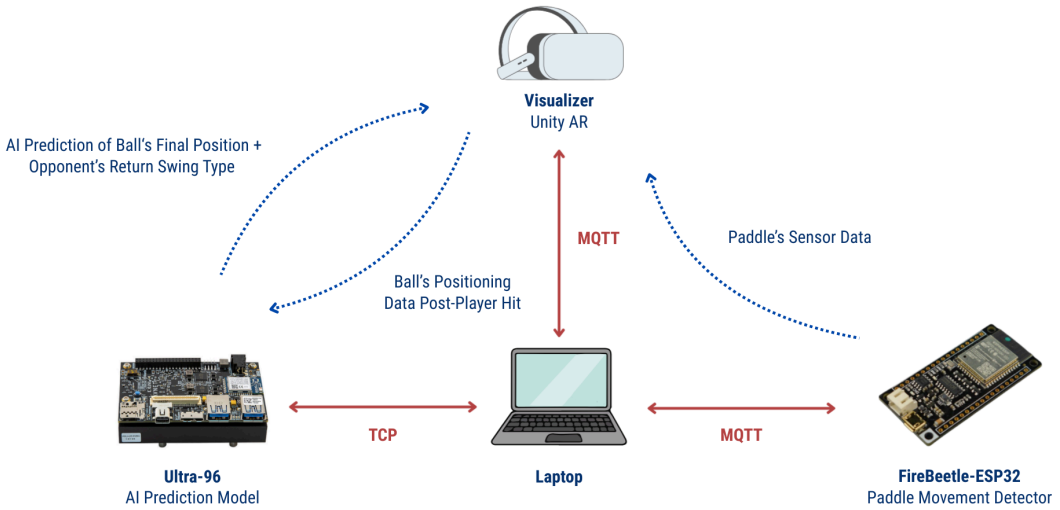
Capacitors	Decoupling / Bulk	Smooths voltage and reduces noise during current spikes	Power rails	Ineffective if undersized or poorly placed

Section 4 Communications

This section is done by Claribel.

4.1 Overview of Communication System

The following diagram provides an overview of the network protocols and data flow between the system components.



All communication between devices is coordinated by a central control unit, namely the laptop. To ensure that the player is able to interact with the game seamlessly in real-time, the communication system prioritises low-latency and reliable data transmission.

Each component is responsible for a crucial, distinct gameplay element that has to be transmitted almost instantly — FireBeetle handles data from the physical paddle; Ultra-96 handles the opponent avatar's return swing; and the Visualiser handles the rendering of the virtual ball and game environment. For the player's immersiveness, delays in communication can greatly affect the game experience.

Additionally, it should be noted that all communication is transmitted securely with TLS encryption.

4.2 Primary Communication Protocol: Message Queuing Telemetry Transport (MQTT)

MQTT is a lightweight messaging protocol that follows a publish-subscribe system. For our system, the laptop is the MQTT broker coordinating messages from and to its two MQTT clients — FireBeetle and Unity Visualizer. To facilitate communication with Ultra96, the laptop is also concurrently a MQTT client (see Section 4.3.4).

We will be using a local MQTT broker hosted on the laptop, where all MQTT client devices will have to be on the same network as the broker. This is possible as during gameplay, the ESP32 (on the player's arm) and the visualizer (on the player's vision) are already in close proximity. Hence, we theorise that the network connection will remain stable, as long as the laptop is nearby.

For all MQTT client connections excluding the Unity visualizer, we are using the Paho MQTT library. This open-source library provides APIs to connect to the broker, publish messages (`client.publish(topic, message)`) and subscribe to topics

(`client.subscribe(topic)`). Moreover, Paho MQTT provides security and reliability, further explained in Sections 4.4 and 4.5.

4.2.1 MQTT Topics

Our communication system is organised using three MQTT topics to synchronise gameplay logic. These are keywords the broker uses to filter messages, and forward to the relevant subscribers.

1) `/paddle`:
    a) *Publisher*: FireBeetle
    b) *Subscriber*: Unity Visualizer
    c) *Data Transmitted*:
        i) Player's intention to serve (signifies match start)
        ii) Player's swing direction, intensity, and racket orientation
        iii) AR user interface interaction controls
2) `/playerBall`:
    a) *Publisher*: Unity Visualizer
    b) *Subscriber*: Laptop (Bridge)
    c) *Data Transmitted*:
        i) Ball's instantaneous position upon player hit [x, y, z]
        ii) Ball's velocity upon player hit [v_x, v_y, v_z]
3) `/opponentBall`:
    a) *Publisher*: Laptop (Bridge)
    b) *Subscriber*: Unity Visualizer
    c) *Data Transmitted*:
        i) Predicted ball's instantaneous position upon AI avatar hit [x, y, z]
        ii) Predicted ball's velocity upon AI avatar hit [v_x, v_y, v_z]
        iii) AI avatar return swing type (0-3)

This separation of gameplay data by topics minimises message congestion by routing data only to relevant subscribers.

4.2.2 MQTT Quality of Service (QoS)

For communication reliability, we rely on MQTT's Quality of Service (QoS) level to enable clients to optimize their network usage. We have pre-selected a QoS level for each MQTT topic, taking into consideration message network bandwidth consumption and message delivery reliability.

In brief, the QoS level allows the broker to set a message's delivery mechanism and re-transmission handling. QoS 0 is on a best-effort basis, with no acknowledgment reply; QoS 1 ensures message delivery with possible duplicates, where the sender re-transmits the message until a PUBACK packet is received; QoS 2 only allows each message to be delivered once and involves a 4-way handshake (QoS 2 PUBLISH, PUBREC, PUBREL, PUBCOMP).

For each MQTT topic, we have determined the following QoS level for data transmission.

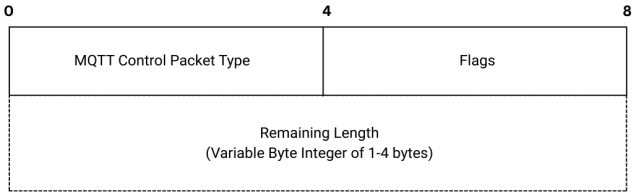
MQTT Topic	Determined QoS Level	Reasoning
`/paddle`	1	Sensor data cannot be lost, but duplicates are tolerable since repeated swing inputs (after a hit) will not interact with the ball and cause unintended gameplay effects
`/playerBall`	2	Ensures Ultra-96 processes the input data (on ball positioning) exactly once. Otherwise, this will result in redundant AI processing by running the same input variables more than once,

		and may possibly output conflicting data that confuses the virtualizer
`/opponentBall`	1	Duplicate AI predictions are acceptable, as the repeated output values will be the same and will not affect the Visualizer gameplay

4.2.3 MQTT Packet Format

There are 15 types of MQTT Control Packets referring to the MQTT documentation. Each packet consists of up to three parts — Fixed Header, Variable Header (Optional) and a Payload (Optional).

The format of the Fixed Header is shown.

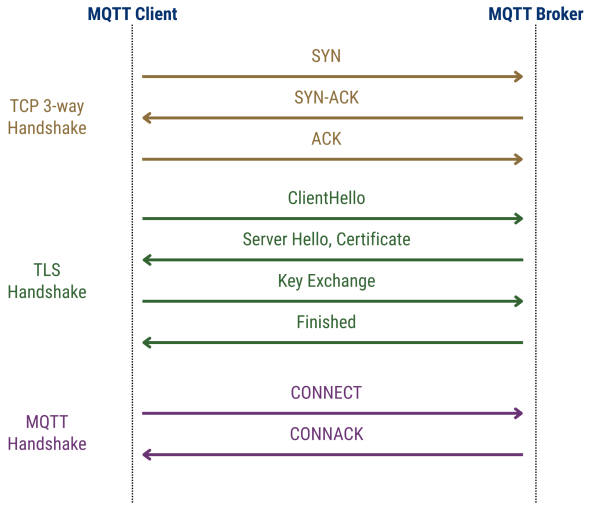


The data from our real-time gameplay is transmitted in the payload part of PUBLISH packets. This payload will be in a JSON format for easy readability for developers and parsing. However, we do aim to test if using alternative encoding formats, such as text data or XML, will result in reduced overhead latency. This is especially since the topics `/playerBall` and `/opponentBall` can be compressed to transmit small numeric arrays. If so, this will improve our communication efficiency and allow for more seamless gameplay.

4.3 Cross-Device Communication

4.3.1 MQTT Connection Setup (Handshaking)

To reiterate, our communication system uses MQTT which operates over TCP/IP, and is encapsulated within TLS for secure communication. When an MQTT client connects to the broker, the following handshakes occur in order:



1) TCP 3-way handshake to establish a TCP connection
    a) Occurs when `client.connect(host, port)`
2) TLS handshake to secure the communication channel
3) MQTT handshake to establish the MQTT session

These handshakes are automatically handled for us and do not need to be manually implemented.

Specifically, during the MQTT handshake, each client initiates a connection with a CONNECT packet to the broker, who will confirm the connection with a CONNACK message. This CONNECT packet importantly contains a unique identifier, `ClientId`, used to identify and track the client state in our system. We will also set the `CleanSession` flag to false to enable persistent sessions (explained in Section 4.5).

4.3.2 FireBeetle and Laptop

The FireBeetle only communicates with the laptop using MQTT (refer to Section 4.3.1 for the handshaking process). This communication is conducted over Wi-Fi, connected to our mobile hotspot.

Some libraries we will use to program the ESP32 include: Wifi.h, PubSubClient.h, WiFiClientSecure.h and ArduinoJSON.h.

The following shows the planned payload format for this communication.

***MQTT Topic*:**	***PUBLISH Packet Payload Format (in JSON)*:**
`/paddle`	`{ "orientation": { "roll": 0, "pitch": 0, "yaw": 0 }, "angularVelocity": { "x": 0, "y": 0, "z": 0 }, "linearVelocity": { "x": 0, "y": 0, "z": 0 }, "isServeAction": false, "buttons": { "up": false, "down": false, "return": false, "select": false } }`

4.3.3 Ultra96 and Laptop

The protocol used for this communication is Transmission Control Protocol (TCP).

Unlike other components (relying on MQTT), the Ultra96 is restricted to the protected "Research-Subnet" connection, as well as shielded by the SoC firewall. Thus, we will be using SSH reverse tunneling (on port 22) to establish this communication. In our system, the laptop acts as a TCP server, while the Ultra96 acts as the TCP client. The laptop has to be first connected to the SoC VPN. With remote port forward (-R flag) the Ultra96's local port 3000 is mapped to the laptop's port 3000, allowing for bidirectional communication.

For consistency with other communication channels, all transmitted data here is serialised as a JSON object.

4.3.4 Ultra96 and Visualizer

The Visualizer is unable to directly communicate with Ultra96 due to the SoC firewall. As mentioned in Section 4.3.3, this will be bypassed with an SSH reverse tunnel, where the laptop acts as a relay node. All data received from the Ultra96 will be published to the MQTT broker by the laptop that will also act as a MQTT client.

***MQTT Topic*:**	***Payload Format in JSON*:**
`/playerBall`	`{ "position": { "x": 0, "y": 0, "z": 0 }, "velocity": { "vx": 0, "vy": 0, "vz": 0 } }`
`/opponentBall`	`{ "position": { "x": 0, "y": 0, "z": 0 }, "velocity": { "vx": 0, "vy": 0, "vz": 0 }, "returnSwingType": 0 }`

The Unity Visualizer uses the M2MQTT library to connect to the broker and send messages. This library provides a `MqttClient` class that makes it easy for connection to the broker by providing its IP address. The Visualizer component can now subscribe to the topic `/opponentBall` with `Subscribe()` method to receive all messages published on that topic; as well as to publish to the topic `/playerBall` with `Publish()` method. Furthermore, this M2MQTT library also supports TLS encryption, ensuring secure communication between the Visualizer and MQTT broker.

4.4 Secure Communication: MQTT over Transport Layer Security (TLS)

We achieve secure communication by encapsulating the MQTT communication within TLS. The Paho MQTT library used for all MQTT client connections supports Transport Layer Security (TLS), where the default secure port for MQTT is 8883. This ensures confidentiality and integrity of the data sent over our communication system. We will also ensure authentication of our MQTT clients by using the API `client.tls_set()`. Each MQTT client in our system will have a unique Client ID for authentication and session management.

For the Ultra96, secure data transmission is implemented with SSH reverse port forwarding. The received data is then securely redistributed via MQTT over TLS to the subscribed clients.

4.5 Handling Reliability Issues

Reliable communication is crucial for our gameplay experience. As mentioned, we are using the Paho MQTT library to handle client connections. This client library has an automatic reconnect functionality that will ensure reliable communication between our broker laptop and clients even when the network connection is interrupted. That is, clients will continue to publish and subscribe to their relevant topics, while the disconnected client attempts to recover.

To further enhance our recovery from lost connections, we will use persistent MQTT sessions. For a disconnected client, the broker will store the client's subscription details, as well as all undelivered messages while the client is offline. This ensures that when the same disconnected client (identifiable by its Client ID) subsequently reconnects to the broker, it is able to resume communication despite the interruption. It is noted that only messages sent with QoS levels 1 and 2 are queued for offline clients.

4.6 Handling Concurrent Communication

As the central coordinating unit, the laptop must simultaneously manage the concurrent data streams between all devices. This includes communication on its own machine, between the locally hosted MQTT broker and its role as a MQTT client bridge (for Ultra96).

Considering the asynchronous nature of our data, we have chosen to use Goroutines where each MQTT topic has its own separate channel to handle transmitting and receiving of messages. This ensures that the communication system remains fluid and does not block for network acknowledgements. We will be using the `paho.mqtt.golang` library to implement this.

Our system spawns independent, lightweight Goroutines to isolate communication. At the current design stage, we expect to have a Goroutine for the following data transmission:

1. Ultra-96 TCP Connection Handler
    a. Dedicated Goroutine to maintain the TCP socket via the SSH reverse tunnel
    b. Isolates the high-latency AI prediction process
2. MQTT Subscriber Handler (one per topic)
    a. Each topic is assigned a `MessageHandler`, which pushes a message into its relevant buffered channels
    b. This ensures our MQTT messages are synchronised

Section 5 Software/Hardware AI

This section was done by Chee Fong.

5.1 Ultra96 synthesis and simulation setup

We will be using the Xilinx Vitis HLS 2022.1 toolchain to synthesize and simulate our neural network accelerator for the Ultra96-V2. This version was selected to ensure full compatibility with the PYNQ v3.0.1 image, which was found when we accessed and checked the Ultra96 server. As this image was built on the 2022.1 software stack, the 2022.1 version was selected to reduce chances of driver mismatches and bitstream parsing errors (Xilinx, 2022).

For the synthesis and verification process, we will first start with C Simulation in Vitis HLS. In this stage, we utilise a C++ testbench to validate the functional logic of our neural network against the golden reference generated from our Pytorch training environment. This allows us to debug the algorithmic correctness before any hardware constraints are applied. Following this, we will perform C Synthesis, where the high-level C++ code will be transferred into Register Transfer Level (RTL) logic. Vitis HLS will apply hardware directives like pipelining and array partitioning at this stage to optimise for latency and resource utilisation (Xilinx, 2022). Following this, it produces a synthesis report that provides preliminary estimates for clock frequency, initiation interval and resource consumption like Block RAM and Digital Signal Processing (DSP) slices.

After C synthesis, C/RTL Co-simulation will be conducted to verify that the C model and the generated RTL has cycle accurate functional equivalence using the same testbench. Timing closure at the target clock (e.g 100 MHz) will be verified later in Vivado after implementation by checking the timing summary metrics such as worst negative slack. This helps to ensure that the data travels fast enough to meet these timing requirements without data corruption or calculation errors. Once there are no problems, this will be packaged as a Vivado IP block for final integration into Ultra96's bitstream.

5.2 Neural Network Explored

For this Pickleball AI, we considered models such as Long Short-Term Memory (LSTM) and Multi-Task Learning (MTL). While LSTMs are commonly used for sequential trajectory data in sports, we decided on an MTL-based Fully Connected Network for our implementation as it offers deterministic latency and higher throughput. Furthermore, this approach reduces the chances of complex feedback loops which are difficult to parallelise on the Ultra96 (Ruder et al., 2017).

For the architecture, we plan to use a Hard Parameter Sharing strategy to maximise resource efficiency in the Ultra96. This will be implemented by having a proposed shared trunk consisting of two dense hidden layers, with an initial target of 64 neurons each, which will learn a generalised representation from the ball's instantaneous position (x, y, z) and velocity (vx, vy, vz). This shared trunk will then branch into two task-specific "heads" to provide comprehensive training feedback.

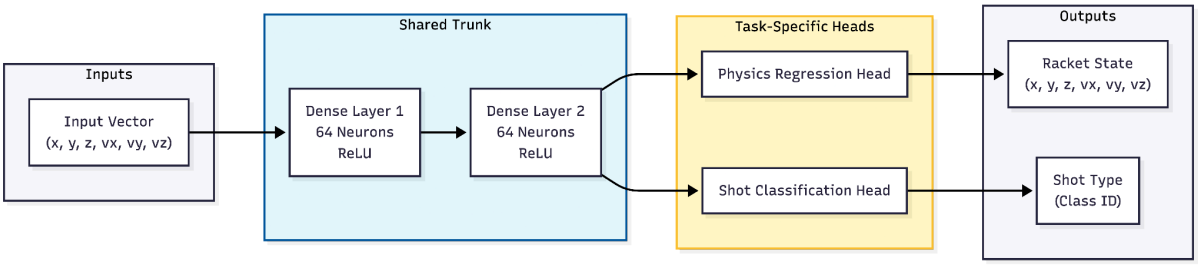
The first head will be a physics regression head that predicts the six element racket state (x, y, z, vx, vy, vz). The second will be a classification head that identifies the shot type class with its class and label shown below:

Class	Label	Shot Description
0	Drive	Fast, low and powerful shot
1	Drop	Soft arching shot
2	Dink	Soft controlled shot

3	Lob	High arching shot

By using this multi-task structure, we expect the model to have better resource utilisation on the FPGA by reusing the same hardware logic for the shared layers while simultaneously providing two outputs. To ensure numerical stability during the transition to hardware, we also plan to use bounded activation functions, in particular, ReLU6, in the trunk to prevent value overflows and a predictable range for fixed-point quantisation (Howard et al., 2017).

A visualisation of the MTL architecture we are planning to implement is shown below:



5.3 Training and testing of model

To ensure that the model is robust and generalisable, we will split it into 3 phases.

For the first phase of training and testing, we will be leveraging on an existing pickleball dataset to establish a baseline model. This will help us to validate our proposed Multi-Task MLP architecture and feature engineering pipeline on real-world trajectory data before committing to our custom data generation. This can help us verify that the selected input features such as ball position and velocity contain sufficient information to predict how the AI should react, allowing us to debug the model early on.

For the second phase, we will transition to a dataset generated via our Unity 3D physics simulation. This will provide thousands of diverse shot values with racket and ball mechanics that match the virtual environment for our AR pickleball application. Unlike real-world data, which suffer from sensor noise or labelling inaccuracies, this pipeline allows us to capture the precise racket state at the moment of impact for every conceivable shot trajectory. To allow for effective learning during the first two phases, a scaled-in/scaled-out strategy will be used where inputs are normalised using a standard scaler and outputs are standardised back to world units.

For the last phase, we plan to incorporate the real-world data captured from our physical pickleball racket. The racket will be equipped with IMU sensors as illustrated in Section 3 to record actual swing dynamics during play. This will help to ensure that the racket movement is accurate to this racket of the live user.

We plan to implement this training pipeline using the PyTorch framework. It was selected for its flexible computational graph and robust support for multi-task loss functions, allowing us to optimise for the Physics Regression Head and Shot Classification head simultaneously. To enhance the training process, we will be using a composite loss function combining Mean Squared Error (MSE) and Cross-Entropy Loss. Furthermore, we intend to integrate the Optuna framework with Pytorch to automate hyperparameter tuning, specifically searching for the optimal learning rate and loss weighing factors (Akiba et al., 2019). These will be all done on a desktop computer with a dedicated GPU for fast and efficient training before transferring to the Ultra96. The final model will be evaluated on a held-out test set using Mean Absolute error (MAE) and Macro F1-Score to ensure precise feedback on the model.

5.4 Implementation of Neural Network Model on FPGA

For the implementation of the Neural Network Model on the FPGA, we will be using a hardware-software co-design approach where the trained model is exported into an HLS-ready format into the Ultra96. This will involve converting the learned weights and biases on from the desktop into constant arrays within C++, while the scalar parameters will be exported using a lightweight JSON artefact. The Ultra96 will implement only the forward pass of the network in fixed-point arithmetic, using selected ap_fixed formats for all internal signals. This separation ensures the hardware datapath will remain compact and deterministic, performing the computationally intensive matrix vector multiplications and activations, while the ARM processor handles the floating-point scaling and data packaging.

To maximise the throughput, the top-level function in Vitis HLS will be synthesised with a AXI4-Lite interface, as the accelerator interface consists of a small number of scalar inputs and outputs, which maps naturally to a register-based design. This exposes the input vectors and output arrays as registers for the ARM processor. We also plan to utilise HLS directives, using "#pragma HLS ARRAY_PARTITION" on activation vectors and weight matrices to increase parallel memory access bandwidth, preventing bottlenecks during computations. Pipelining will also be applied to compute loops using "pragma HLS PIPELINE" to allow the design to accept new operations every clock cycle where possible.

On the Ultra96 itself, the PYNQ runtime will be used to manage the accelerator using a five step process:

1. Performing standard scaling on input data using the JSON parameters
2. Writing the six scaled features to the IP registers
3. Reading back the scaled regression outputs and classification logits
4. Performing inverse scaling to convert back the values
5. Applying an argmax function to determine final shot type

5.5 Evaluation of Timing, Power and Area of Design

Timing, power and area will be evaluated at two distinct levels.

Firstly, HLS estimations in the Vitis HLS will be used to obtain preliminary data on latency, initiation interval and estimated resource usage. These metrics will allow us to rapidly iterate to meet our target clock frequency.

Secondly, post-implementation Vivado reports can be used to access the final area and timing closure. We plan to analyse the utilisation report for actual post-routing resource consumption and the timing summary report to verify that the worst negative slack meets the clock constraints, ensuring the design runs reliably without setup or hold violations.

Thirdly, for power consumption, we aim to employ Vivado power analysis to estimate the static and dynamic power based on the implemented netlist and switching activity assumptions. If a more representative estimation is required we will cross check these results using the Xilinx power Estimator workflow, utilising our post-implementation utilisation data as inputs.

Finally, to tie these metrics back to our dataset, we will run a batched interface on the Ultra96 using held out samples. By measuring the end-to-end interface latency on the ARM side and combining it with our power estimates, we can estimate the relative energy efficiency per inference. This provides a practical metric to compare different HLS directive sets and clock targets, ensuring the system can produce consistent, low-latency feedback for the software visualiser.

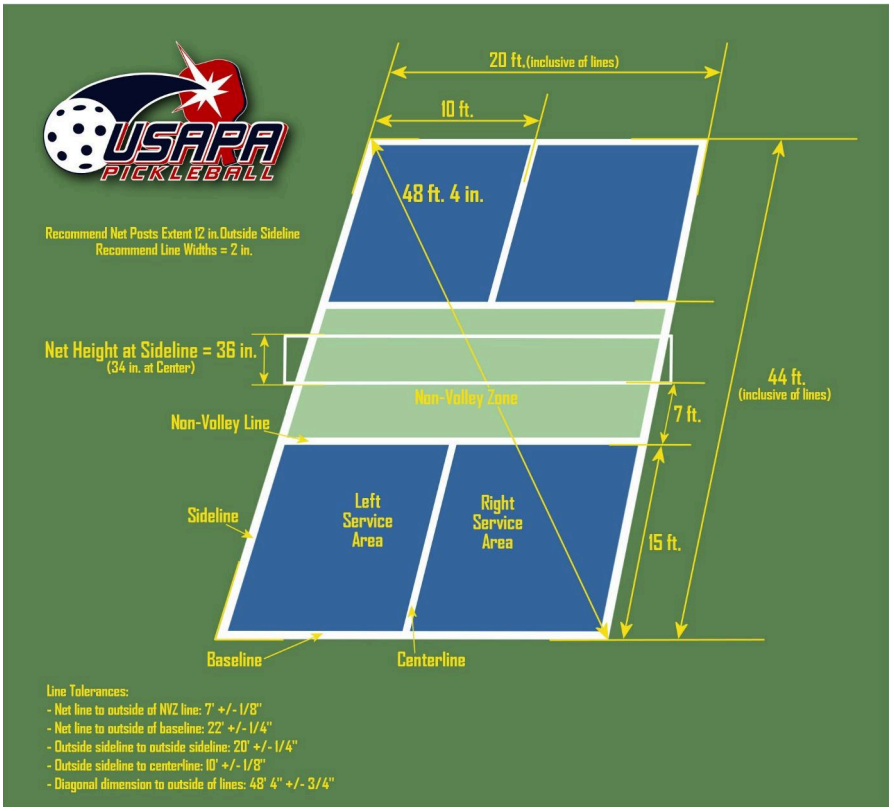
Section 6 Software Visualizer (Unity-based AR Application)

This section is done by Khanh.

6.1. Visualizer Design

The Software Visualizer is a Unity-based mobile augmented reality (AR) application that delivers a first-person pickleball gameplay and training experience on a smartphone. Using the device camera feed as the background, the Visualizer renders the virtual court, ball, paddle, opponent, and gameplay telemetry into the user's physical space. Its role in the overall system is to (i) provide an immersive AR scene, (ii) mirror the player's real paddle motion with low latency, (iii) simulate ball motion and interactions using Unity physics, and (iv) display real-time feedback such as score and rally progression.

To approximate a real pickleball setup, the Visualizer generates a standard pickleball court and net with real-world dimensions, including a 44 ft × 20 ft court and a net height of 36 in at the sidelines and 34 in at the center. The sub-areas of the court (service areas, centerline, baseline, non-volley zone) are visualized as part of the AR court to maintain realism and support consistent spatial orientation during gameplay.



The gameplay is designed as a one-player, first-person rally against an AI-powered opponent. In the AR scene, the user plays from one side of the court while a default human-like opponent avatar stands on the opposite side holding a paddle. The opponent's return shots are generated by the AI model discussed in 5.2 Neural Network Explored and 5.3 Training and testing of model, and the Visualizer is responsible for rendering these returns in a way that remains visually and physically consistent with the player's actions and the simulated ball state.

The Visualizer supports four standard pickleball shot types—Drive, Drop, Dink, and Lob—for both the player and the AI opponent. The ball trajectory after contact is determined primarily by the paddle–ball contact position and the post-contact ball velocity vector, so the motion remains governed by the physics simulation in Unity. While ball spin can affect real

trajectories, spin is intentionally excluded in the first integration to reduce complexity and stabilize the end-to-end system; this allows the project to prioritize reliable swing-to-trajectory behavior before extending to angular dynamics.

For user experience, the application is deployed on iOS and runs on a groupmate's iPhone. During play, the phone can be used handheld, or mounted in a head-mounted phone holder to provide a stable, hands-free first-person AR viewpoint. Paddle motion data (e.g., position/orientation and derived velocity) is transmitted from the sensor hardware attached to the paddle to the Unity application through the network communication pipeline. This enables the Visualizer to update the virtual paddle pose and ball interaction in real time.

6.2 Phone Placement, Displayed Data, and Design Constraints

The Visualizer runs on an iPhone and supports handheld use or mounting in a phone head-mount holder to stabilize the first-person viewpoint. In both modes, the AR camera pose is the reference for what the player sees, and the gameplay UI remains minimal to avoid covering the real environment.

The display contains two layers: (i) world-space AR objects (court, net, ball, opponent, paddle) anchored to the plane, and (ii) a screen-space UI overlay for score, rally state, and lightweight feedback prompts. The main constraints are (a) AR anchoring drift (especially on low-texture floors), (b) end-to-end latency (paddle motion → rendered paddle → hit outcome), and (c) network jitter from MQTT delivery, which can cause visible discontinuities if not smoothed on the Unity side.

6.3 Communication and Data Flow (Aligned with Section 4 Comms)

The Visualizer acts as an MQTT client and follows the exact topic separation defined in Section 4. It subscribes to `/paddle` from the FireBeetle to receive the player's orientation (roll/pitch/yaw), angular velocity, linear acceleration, serve intent (`isServeAction`), and UI button inputs. It publishes `/playerBall` only on confirmed paddle–ball contact, containing the ball position and velocity immediately after the hit, and it subscribes to `/opponentBall` containing the AI-generated return ball state and `returnSwingType`.

Because Unity gameplay updates must occur on the main thread, incoming MQTT messages are buffered in a thread-safe queue and consumed during the Unity update loop. For high-rate `/paddle` messages, the Visualizer can safely drop older buffered poses and always apply the most recent sample to minimize perceived lag. In contrast, `/playerBall` and `/opponentBall` messages are processed event-by-event because they change the rally state and should not be skipped.

6.4 Visualizer Software Architecture (Frameworks, Modules, and Streamed Inputs)

On iOS, the AR stack is built with Unity's AR pipeline (e.g., AR Foundation with ARKit). Plane detection and raycasting determine where the court anchor should be placed. Once placed, the anchor transform becomes the stable reference frame: all court geometry, the ball, and the opponent are parented under this anchor so that minor camera motion does not desynchronize the gameplay world.

The Visualizer runtime is separated into a small set of Unity components. The AR Session module handles plane detection and anchor placement. The Network module manages MQTT connection, subscription, and message buffering. The Paddle module converts `/paddle` inputs into a usable virtual paddle pose. The Ball Physics module owns the Unity Rigidbody/collider setup and computes post-hit ball velocity. The Opponent module consumes `/opponentBall` and drives both ball state application and opponent animation using `returnSwingType`. Finally, the Game State module enforces serve/rally transitions, officiating rules, and UI telemetry updates.

A key technical point is that the FireBeetle payload provides orientation and motion signals rather than an absolute 3D position. To produce a stable "virtual paddle pose" in AR, the Visualizer uses a hybrid approach: paddle position is approximated relative to the camera (a fixed offset representing a right-handed grip position), while paddle rotation is taken from the IMU orientation. Swing speed used for ball impact can be estimated from IMU angular velocity and short-window motion features (rather than long-term acceleration integration, which drifts). This matches the system goal of low-latency mirroring without requiring full external tracking.

6.5 Overlaying AR Information on the Camera Feed

Unity renders the live camera feed via the AR camera background, while all gameplay objects are rendered in world space under the anchor. This keeps the court visually "on the floor" and allows the ball to move naturally through the AR scene.

Telemetry (score, rally state, and simple prompts) is drawn using a screen-space Canvas so it remains readable even if AR tracking briefly jitters. This avoids placing text in world space where it can become unstable or hard to read due to depth changes, lighting variation, or camera motion.

6.6 Enabling AR of Gameplay Events (Serve, Hit, Return, Scoring)

A rally begins only when the Visualizer receives a valid serve intent (`isServeAction`) together with a consistent forward-swing signature from `/paddle` (as established in the hardware intent design). On serve start, the ball is spawned at a deterministic starting location relative to the player side of the anchored court and given an initial velocity consistent with the serve gesture.

During the rally, paddle–ball contact is detected using Unity colliders (paddle collider against ball Rigidbody collider). On contact, the Visualizer computes the post-hit ball velocity using the paddle orientation, the contact point, and a swing-speed estimate derived from IMU angular velocity (and other short-window features if required). The resulting ball state is applied immediately in Unity, and the same state is published to `/playerBall` to trigger AI inference.

When `/opponentBall` arrives, the Visualizer applies the returned ball position/velocity as the opponent's hit outcome (i.e., the AI is specifying the return ball state, not merely "predicting" it). The opponent avatar animation is selected using `returnSwingType` so the visuals remain consistent with the return category, while the ball motion remains governed by Unity physics after that instant.

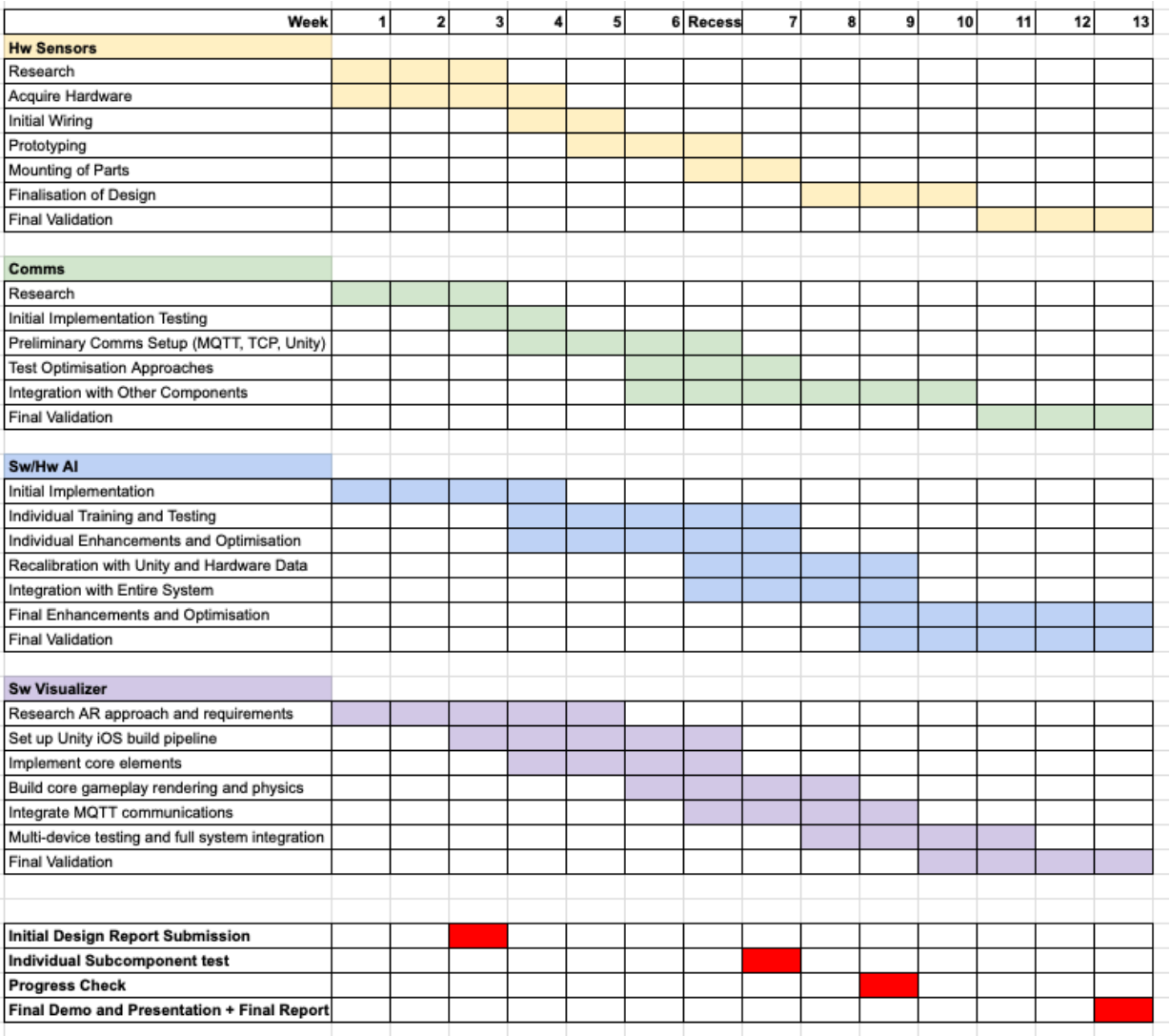
6.7 Implemented Gameplay Rules (Scoring Scope for Automated Officiating)

To make automated officiating unambiguous, the Visualizer will implement an explicit scoring model that is simple enough for reliable AR play yet representative of real matches. A practical scope is best-of-3 sets, with each set played to 11 points and win-by-2, using rally scoring so every rally produces a point outcome that the Visualizer can update immediately alongside the AR telemetry. This definition is intentionally brief (not a full pickleball tutorial), but it clearly states what the system enforces when it "automatically validates shots and determines rally outcomes," and it matches the report's emphasis on continuous, fluid play without manual arbitration.

Section 7 Project Management Plan

This section was a joint effort by all members.

The plan for developing the Pickleball AR application is outlined in the following Gantt chart.



References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623–2631. https://doi.org/10.1145/3292500.3330701

All Star Tennis Supply. (2019, September 13). *Everything you need to know about pickleball courts and equipment*. https://www.allstartennissupply.com/blog/everything-you-need-to-know-about-pickleball-courts-and-equipment/

DFRobot. (n.d.). *FireBeetle Board-ESP32 (V5.0)* [Datasheet]. RS Components. https://docs.rs-online.com/308c/A700000008916343.pdf

Electronics. (2020, October 28). *MOSFET motor control* [Video]. YouTube. https://www.youtube.com/watch?v=OID9lsts-PE

Espressif Systems. (2023). *ESP32 series datasheet* (Version 4.3). https://documentation.espressif.com/esp32_datasheet_en.pdf

Espressif Systems. (2024). *ESP32-WROOM-32 datasheet* (Version 3.4). https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf

Flamm, J. (2025, January 7). *Pickleball participation surges to nearly 20 million in 2024*. The Dink. https://www.thedinkpickleball.com/report-pickleball-participation-surges-to-nearly-20-million-in-2024/

Foolish Engineer. (2025, May 25). *How to STOP inrush current using P-channel MOSFET? Inrush current limiter circuit | N-MOS circuit* [Video]. YouTube. https://www.youtube.com/watch?v=8z8dK6AFF5M

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient convolutional neural networks for mobile vision applications*. arXiv. https://arxiv.org/abs/1704.04861

Liam, E., & Mohan, M. (2025, November 6). *Singapore's pickleball boom brings growing pains*. CNA. https://www.channelnewsasia.com/singapore/pickleball-growing-pains-limited-space-courts-5321881

Omron. (n.d.). *B32: Tactile switch key top* [Datasheet]. https://omronfs.omron.com/en_US/ecb/products/pdf/en-b32.pdf

Omron. (n.d.). *B3F: Tactile switch* [Datasheet]. https://omronfs.omron.com/en_US/ecb/products/pdf/en-b3f.pdf

Panasonic. (2010). *NCR18650B lithium ion battery specification* [Datasheet]. Shoptronica. https://www.shoptronica.com/files/Panasonic-NCR18650.pdf

Pickleball Athletic Club. (2025, July 28). *How long does a pickleball game last? (15-75 minutes)*. https://playatpac.com/how-long-does-pickleball-game-last/

PlantUML. (n.d.). *Sequence diagram*. https://plantuml.com/sequence-diagram

Reddit. (2026). *Comparison best 9 axis IMU sensors*. https://www.reddit.com/r/esp32/comments/1k4rttu/comparison_best_9_axis_imu_sensors_bmi160bmm150/

Ruder, S. (2017). *An overview of multi-task learning in deep neural networks*. arXiv. https://doi.org/10.1706/05098v1

Seeed Studio. (n.d.). *TTP223 touch sensor* [Datasheet]. https://files.seeedstudio.com/wiki/Grove-Touch_Sensor/res/TTP223.pdf

SlimeVR. (2025). *IMU comparison*. https://docs.slimevr.dev/diy/imu-comparison.html

STMicroelectronics. (2019). *LSM6DSR: iNEMO inertial module, always-on 3D accelerometer and 3D gyroscope* [Datasheet]. https://www.st.com/resource/en/datasheet/lsm6dsr.pdf

stm32duino. (2024). *LSM6DSR* [Computer software]. GitHub. https://github.com/stm32duino/LSM6DSR

Sun, D. (2025, September 26). *Over 700 pickleball noise complaints in HDB estates since 2024*. The Straits Times. https://www.straitstimes.com/singapore/politics/over-700-pickleball-noise-complaints-in-hdb-estates-since-2024

Texas Instruments. (2015). *INA219 zero-drift, bidirectional current/power monitor* [Datasheet]. https://www.ti.com/lit/ds/symlink/ina219.pdf

Xilinx. (2022). *PYNQ: Python productivity for Zynq*. https://pynq.readthedocs.io/en/latest/

Xilinx. (2022). *Vitis High-Level Synthesis user guide (UG1399, v2022.2)*. https://www.xilinx.com/support/documents/sw_manuals/xilinx2022_2/ug1399-vitis-hls.pdf